

LaosSA: An Approach Based on Fine-tuning Pre-trained Language Models for Lao Sentiment Analysis

Khamtay Kongmanh¹, Quang-Vinh Pham², Quang-Hung Le^{1*}

¹ Department of Information Technology, Quy Nhon University, Gia Lai, Vietnam
kongmanh4551050274@st.qnu.edu.vn, lequanghung@qnu.edu.vn

² Department of Data Science, TMA Tech Group, Gia Lai, Vietnam
pqvinh@tma.com.vn

Abstract

Effective sentiment analysis for the Lao language is hindered by a scarcity of annotated resources and the inherent noise of real-world user feedback, which often features code-mixing and informal script patterns. This study addresses these challenges by introducing a high-quality, filtered benchmark of 25,139 Lao-language reviews collected from popular digital applications. We evaluate classical baselines, a lightweight textual convolutional neural network model trained from scratch, and several state-of-the-art pre-trained language models, including XLM-RoBERTa, multilingual BERT, and a Lao-specific multilingual BERT, across full fine-tuning, low-rank adaptation, and k-fold cross-validation settings as applicable. Our experimental results demonstrate that XLM-RoBERTa achieves superior performance with 96.64% accuracy and an F_1 -macro score of 0.9617. The textual convolutional neural network also provides a strong neural baseline with 94.03% F_1 -macro on the fixed split, placing it between the sparse baselines and the strongest pre-trained language model variants. We also find that while Lao-oriented pre-training provides significant benefits, low-rank adaptation offers a highly efficient performance-cost trade-off, maintaining competitive accuracy with minimal trainable parameters. Our research highlights the robustness of multilingual transfer for low-resource sentiment classification and provides a publicly available dataset and code at <https://github.com/KT246/LaosSA>.

Index Terms

Lao Sentiment Classification, Pre-trained Language Models, TextCNN, LoRA, Low-resource NLP.

1 Introduction

Sentiment analysis stands at the core of the contemporary digital economy; it now can be used by service providers to extract actionable insights from the immense volume of unstructured comments received from online marketplaces and digital-service services [1]–[3]. Automatic classification of the user sentiment is an important but underrepresented frontier for the Lao digital ecosystem. While AI-based sentiment classification in Lao would enable scalable analysis of customer satisfaction across app stores and social media, it has not been able to replicate any large-scale analysis because of the dearth of annotated corpora, standardized benchmarks, and mature Natural Language Processing (NLP) toolchains [4]–[6]. The resource gap is further compounded by the inherent linguistic complexities of the Lao script. As a continuous, non-linear script (an abugida) lacking explicit word delimiters, Lao presents significant challenges to standard tokenization and morphological parsing [5]. Moreover, real-world digital discourse in Lao is often noisy and filled with high levels of semantic noise, colloquialism, and code-mixed patterns [7]–[9], making it complicated to directly transfer training models on formal or high-resource data sets. In this way, methodologies optimized for high-resource languages often fail to capture the subtle polarity of Lao mobile application reviews. However, traditional rule-

based and classical machine learning methods sometimes suffer the same shortcomings and often require extensive feature engineering: they lack contextual depth. This has led to a rise in Pre-trained Language Models (PLMs). However, generalized multilingual PLMs often underrepresent low-resource languages and hence suffer from sub-optimal representation mappings and tokenization inefficiencies [4]. This reinforces the call for a serious comparative study of the state-of-the-art multilingual PLMs (XLM-RoBERTa [10], mBERT [11]), Lao-specific models (mBERT-Lao [4]), and lightweight neural and sparse baselines. We propose that the tradeoff between Lao focused pre-training and multilingual transfer is superior, and consider tuning the parameters of adaptive model, such that they can maintain these advantages at a fraction of the input processing costs. Let us introduce *LaosSA*, a complete benchmark for the classification of Lao sentiment, using a filtered dataset of 25,139 reviews of mobile applications. Key insights: Our contributions include these:

- We present a high-quality Lao sentiment dataset, manually annotated according to the qualitative characteristics, which was specifically curated from Mobile Application reviews to learn from actual, the real colloquial script and code mixing conditions.
- We also set up a high-performance benchmark for different modeling pipelines to check cross-lingual transfer, monolingual pre-training, Low-Rank Adaptation (LoRA) [12] and Textual Convolutional Neural Network (TextCNN) from scratch neural training.
- We conduct multi-dimensional analyses of predictive performance, error patterns, and training efficiency, thereby offering actionable insights into the trade-off between model quality and computational resource requirements for Lao NLP.

The rest of this paper is structured as follows. Section 2 situates the study within the broader low-resource sentiment analysis and PLM landscape as it relates to broader trends. The construction of the LaosSA dataset and the steps of data collection, annotation and cleaning pipeline is described in Section 3. Section 4 formalizes the classification task and introduces the modelling framework and their refinement steps of fine-tuning. Section 5 presents our configurations of experiments, results and critical discussion. Section 6 ends the paper and presents potential future research directions.

2 Related Work

The evolution of Lao sentiment analysis has followed the broader trajectory of PLM development and low-resource adaptation, spanning general multilingual baselines and dedicated Lao-specific benchmarks.

Early progress was marked by the introduction of multilingual BERT (mBERT) in 2019 [11], which provided the first massive transformer pre-training for over 100 languages, including zero-shot capabilities for low-resource scripts. However, a major limitation of mBERT remains its sub-optimal representation of Lao due to a shared vocabulary that assigns limited capacity to low-resource languages, often resulting in fragmented subword tokens for continuous scripts. This was partially addressed by XLM-RoBERTa (XLM-R) in 2020 [10], which scaled up training data and refined cross-lingual representations. While XLM-R demonstrates significantly stronger zero-shot and few-shot performance, its primary limitation in the Lao context remains its generic pre-training objective, which does not explicitly account for the specific character boundaries or the noisy, informal code-mixing prevalent in modern digital communication.

A closely related regional example is Vietnamese e-commerce sentiment classification, where recent work reports strong results from fine-tuning PhoBERT, viBERT, and XLM-RoBERTa variants while also highlighting informal language, code-mixing, and domain-adaptation challenges that are relevant to our setting [3].

Low-resource language processing is still constrained by the lack of large corpora, annotated benchmarks, and mature preprocessing tools. For Lao, this limitation has been noted in work on word segmentation, text classification, and broader resource construction

[4], [5]. Because Lao is written as a continuous script, preprocessing itself can become a source of error: tokenization and boundary detection are less straightforward than in whitespace-delimited languages [5]. This challenge is directly relevant to the current dataset, where many user reviews are very short and noisy.

The landscape shifted in 2022 with the introduction of LaoPLM [4], the first dedicated Lao-specific pre-trained model. Its strength lies in its ability to outperform generic multilingual models on formal Lao text classification by leveraging monolingual pre-training. Nevertheless, the evaluations in LaoPLM were constrained by relatively small datasets and formal registers, leaving the task of analyzing noisy, colloquial mobile app reviews largely unaddressed. More recently, in 2025, efforts by Nguyen et al. [5] established a semi-supervised word-segmented corpus using large language models. While this work successfully expanded the dataset resources for Lao, the focus remained on categorization and resource generation rather than on the nuanced polarity shifts found in short-text sentiments.

PLMs are now the dominant backbone for text classification because they provide contextualized representations that can be adapted across tasks [13], [14]. For Lao, the distinction between monolingual and multilingual PLMs is particularly important [4]. A Lao-oriented model can better reflect local tokenization behavior and lexical patterns, whereas multilingual models benefit from broader pretraining but may devote less capacity to low-resource languages [4], [13], [15].

Recent low-resource sentiment studies still report clear gains from transformer-based models, but they also show that performance depends on language fit, adaptation strategy, and dataset characteristics [7]–[9]. This is why our experiments compare XLM-RoBERTa and mBERT with mBERT-Lao, while also retaining a lightweight TextCNN and strong sparse baselines in the benchmark.

Our work (LaosSA, 2026) represents the next milestone by addressing these gaps across three dimensions: scale, domain, and adaptation strategy. Unlike prior benchmarks, LaosSA provides a large-scale (25,139 samples) filtered dataset specifically curated from mobile application reviews, capturing real-world noise and code-mixed patterns. Furthermore, we evaluate both full fine-tuning and Parameter-Efficient Fine-Tuning (PEFT) via LoRA to establish practical deployment baselines that were previously underexplored for the Lao digital ecosystem.

3 Dataset Construction and Analysis

This section details the construction and systematic refinement of the Lao sentiment benchmark, covering data collection from digital-service platforms, the manual annotation process, and the multi-stage filtering pipeline designed to ensure dataset quality.

3.1. Data Collection

The reviews used in this study were collected from Google Play applications that are widely used in Laos. The corpus includes comments from banking apps, social media apps, and other digital-service applications rather than from a single e-commerce marketplace. This choice was practical: these applications produced enough Lao user feedback to build a usable sentiment dataset, and they reflect the short mobile reviews that a deployed Lao sentiment system would need to handle.

Each record contains two fields, namely the review text and a binary sentiment label. The final benchmark also retains short code-mixed reviews in which Lao words appear together with English interface terms such as *app*, *bug*, or *OK*, provided that the overall sentiment remains interpretable. This choice keeps the released benchmark closer to real mobile feedback in Laos and directly corresponds to the code-mixing setting referenced in the released resources.

3.2. Annotation Scheme

The raw reviews were manually annotated with binary sentiment labels. Each review was assigned a single label based on its overall polarity: label 1 for positive sentiment

and label 0 for negative sentiment. The annotation process focused on the dominant sentiment expressed in each review, including explicit praise, complaints, dissatisfaction, or statements about overall user experience.

To improve label consistency, the annotation followed a simple set of practical guidelines. Reviews expressing clearly favorable opinions toward an application, service quality, or user experience were marked as positive, whereas reviews expressing frustration, malfunction, dissatisfaction, or criticism were marked as negative. During dataset refinement, noisy, invalid, or suspicious samples were revisited through the cleaning pipeline described below so that only reliable sentiment instances were retained in the final corpus.

3.3. Data Preprocessing and Cleaning Pipeline

Before final training, the raw reviews were cleaned and filtered in several stages. After manual labeling, we removed noisy content such as repeated characters, emojis, invalid symbols, and strings that did not form meaningful Lao text. Particular attention was paid to Lao-specific character errors, especially cases in which vowels appeared without consonants or in which character combinations were visually present but linguistically invalid.

To assess label quality more efficiently, we trained a lightweight screening model and compared its predictions with the manual annotations. Samples that repeatedly agreed with their labels were retained, whereas mismatched cases were rechecked and many were removed. This step served as a practical filter for suspicious or low-quality reviews before the final benchmark experiments.

We also used a separate 5-fold cross-validation auditing step for difficult cases during dataset cleaning. This step was part of data filtering only and is distinct from the 3-fold cross-validation used later for model evaluation. Reviews that were repeatedly misclassified across folds were treated as unstable, and many of them contained invalid Lao character patterns, lexical inconsistencies, or meaningless text fragments.

After this filtering process, the remaining reviews formed the final dataset used for training and validation.

3.4. Dataset Statistics

Table I summarizes the final train and validation splits after cleaning and filtering. The two partitions remain closely matched in class balance and contain no missing text or labels. The reviews are short on average, with only a small number of much longer comments, which is consistent with the brief and noisy style of app-store feedback.

TABLE I
FINAL TRAIN AND VALIDATION SPLIT STATISTICS FOR THE LAO REVIEW BENCHMARK AFTER FILTERING AND PREPROCESSING.

Statistic	Train	Validation
Rows	20,111	5,028
Columns	2	2
Empty text	0	0
Empty label	0	0
Unique text	20,095	5,028
Duplicated text within split	16	0
Negative reviews (Label = 0)	6,312 (31.39%)	1,578 (31.38%)
Positive reviews (Label = 1)	13,799 (68.61%)	3,450 (68.62%)
Minimum text length (characters)	2	3
Average text length (characters)	29.41	30.29
Maximum text length (characters)	618	506
Minimum word count	1	1
Average word count	1.52	1.52
Maximum word count	66	77

4 Methodology

This section presents the formal task definition for Lao sentiment classification, introduces the selected PLMs, and describes the fine-tuning strategies and training objectives employed in our modeling framework.

4.1. Problem Definition

We formulate Lao sentiment classification as a supervised binary text classification task over cleaned Lao reviews. Let the benchmark dataset be defined as

$$\mathcal{D} = \{(x_i, y_i)\}_{i=1}^N, \quad (1)$$

where each review $x_i = (w_1, w_2, \dots, w_n)$ is represented by the token sequence produced by a given tokenizer, and each label satisfies $y_i \in \{0, 1\}$, with 0 denoting negative sentiment and 1 denoting positive sentiment. Each review receives a single sentence-level label corresponding to its dominant polarity, even when the text remains short, informal, or noisy.

The task is to learn a classifier f_ψ that maps a Lao review to one of the two sentiment classes. For each input x , a model estimates a posterior distribution over the label space and predicts the dominant polarity according to

$$\hat{y} = f_\psi(x) = \arg \max_{c \in \{0,1\}} p_\psi(y = c \mid x), \quad (2)$$

where ψ denotes the parameters of either a classical classifier or an adapted PLM-based classifier, and c indexes the candidate sentiment class. This formulation keeps the task definition model-agnostic so that sparse baselines and PLM-based systems can be compared under the same prediction rule.

The benchmark is defined under several practical assumptions. Each review is treated as an independent sample without conversational context or user metadata. The experiments focus on document- or sentence-level polarity classification rather than aspect-level sentiment extraction, and all systems operate directly on the cleaned review text rather than on hand-crafted linguistic features. These assumptions keep the comparison centered on representation quality and adaptation strategy rather than on feature design or external metadata.

The evaluation protocol follows the final filtered split described in Section 3.4. For the main benchmark, models are trained on 20,111 training samples and evaluated on a fixed validation set of 5,028 samples. In addition, a separate 3-fold cross-validation setting is used as a robustness check on the training partition. Performance is reported using Accuracy, Precision-macro, Recall-macro, and F_1 -macro, with F_1 -macro treated as the primary comparison metric because of the moderate class imbalance between positive and negative reviews.

4.2. Pre-trained Language Models

We evaluate three PLMs: XLM-RoBERTa [10], mBERT [11], and mBERT-Lao [4]. XLM-RoBERTa and mBERT serve as multilingual references for cross-lingual transfer [13]–[15], while mBERT-Lao provides a Lao-supporting alternative [4]. Table II summarizes the main properties of the three models under the benchmark-facing names used throughout this paper. All PLM runs share the same preprocessing and evaluation protocol, and the LoRA setting attaches adapters to the query, key, and value projections.

TABLE II
PRE-TRAINING AND ARCHITECTURAL COMPARISON OF THE PLMS USED IN THE BENCHMARK.

Aspect	XLM-RoBERTa	mBERT	mBERT-Lao
Language scope	Multilingual (100 languages)	Multilingual (104 languages)	Lao-supporting
Pre-training data	CommonCrawl multilingual corpus (~2.5 TB)	Multilingual Wikipedia corpus	Lao-oriented RoBERTa-style pretraining
Tokenizer	SentencePiece, ~250k subwords	WordPiece, ~119k vocabulary	RoBERTa tokenizer, ~50k vocabulary
Architecture (base)	12 layers, 768 hidden, 12 heads	12 layers, 768 hidden, 12 heads	12 layers, 768 hidden, 12 heads
Role in this benchmark	Strong multilingual reference	Strong multilingual reference	Lao-supporting comparison model

In addition to the PLMs, the benchmark includes four non-PLM references: Logistic Regression, Support Vector Machine (SVM), Decision Tree [16], and TextCNN [17]. The first three use Term Frequency-Inverse Document Frequency (TF-IDF) character n -gram features, while TextCNN is evaluated under from-scratch training as a compact convolutional classifier with 128-dimensional embeddings, 128 filters per kernel size $\{3, 4, 5\}$, and dropout 0.3. This gives the benchmark a lightweight neural baseline that does not rely on pretrained language-model weights and allows for a direct comparison between transfer-learning and static initialization paradigms.

4.3. Fine-Tuning and From-Scratch Training Strategies

Each PLM is trained with either Full Fine-tuning or LoRA, while the neural baseline is trained from-scratch. Full Fine-tuning updates the backbone and classification head end to end. LoRA instead freezes the pretrained weights and learns low-rank updates in selected attention projections [12], [18], [19]. From-scratch training involves random weight initialization and localized optimization without knowledge transfer. We also report 3-fold cross-validation as a robustness check; in that setting, a separate model is trained on each fold and the final score is computed from the concatenated out-of-fold predictions.

Let θ denote the backbone parameters and ϕ the classification head. Full Fine-tuning solves

$$(\theta^*, \phi^*) = \arg \min_{\theta, \phi} \mathcal{L}_{\text{train}}(\theta, \phi). \quad (3)$$

For LoRA, the pretrained matrix is decomposed as $W = W_0 + BA$, where W_0 is frozen and $A \in \mathbb{R}^{r \times d}$ and $B \in \mathbb{R}^{d \times r}$ are trainable. The optimization becomes

$$(A^*, B^*, \phi^*) = \arg \min_{A, B, \phi} \mathcal{L}_{\text{train}}(W_0 + BA, \phi). \quad (4)$$

For cross-validation, out-of-fold predictions from all K folds are concatenated and scored jointly. The reported score is therefore

$$\text{Score}_{\text{CV}} = \text{Score} \left(\bigcup_{k=1}^K \hat{D}_{\text{val}}^{(k)} \right), \quad K = 3, \quad (5)$$

where $\hat{D}_{\text{val}}^{(k)}$ denotes the predictions produced by the best checkpoint of fold k on its held-out partition.

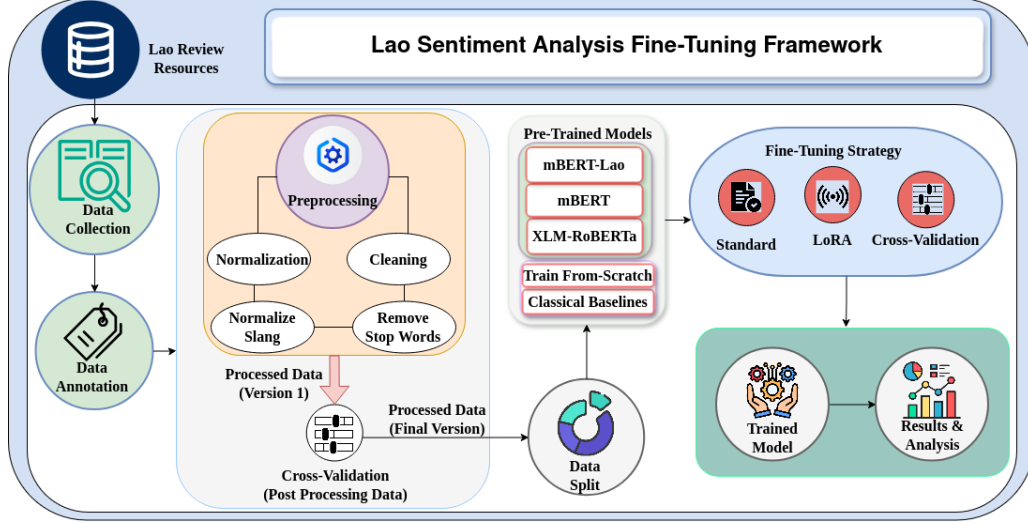


Fig. 1. Overall methodology framework for Lao sentiment classification, including dataset preparation, model selection, and training strategies.

Fig.1 summarizes the benchmark in three stages. The workflow begins with Google Play review collection, binary annotation, normalization, cleaning, and the post-processing audit used to construct the final split. It then trains XLM-RoBERTa, mBERT, mBERT-Lao, TextCNN, and TF-IDF baselines under the strategy variants applicable to each family, including Full Fine-tuning, LoRA, from-scratch training, and 3-fold cross-validation. Finally, the pipeline exports checkpoints, validation predictions, standard metrics, timing summaries, and false-positive/false-negative artifacts for the later analysis sections.

4.4. Training Objective

PLM runs minimize weighted cross-entropy loss. Class weights compensate for the moderate label imbalance in the final dataset. For a training set of N samples and two sentiment classes, the objective is

$$\mathcal{L} = - \sum_{i=1}^N \sum_{c=1}^2 w_c y_{i,c} \log \hat{y}_{i,c}, \quad (6)$$

where w_c is the weight for class c , $y_{i,c}$ is the one-hot target, and $\hat{y}_{i,c}$ is the predicted probability for class c .

We train the PLMs with standard pretrained-model tooling and fine-tuning practice [13], [14], using a learning rate of $1e^{-5}$, weight decay 0.01, warmup ratio 0.1, and maximum sequence length 128. For Full Fine-tuning and LoRA, the best checkpoint is selected by validation loss. TextCNN uses the same epoch budget, batch sizes, and sequence length, but it initializes all parameters from scratch. The classical baselines use the same split and train TF-IDF character n -gram features from 3 to 5 with `min_df=2` and at most 50,000 features before classifier fitting.

Algorithm 1 summarizes the unified benchmark pipeline used throughout the experiments. It reflects the fixed-split baseline runs, the PLM-based fixed-split runs, and the separate 3-fold cross-validation setting reported later in Section 5.

Algorithm 1 Unified training and evaluation pipeline for Lao sentiment classification

Require: fixed-split $D_{\text{train}}, D_{\text{val}}$, PLM set $\mathcal{M}$, baseline set $\mathcal{B}$, neural reference set $\mathcal{N}$, strategies $\mathcal{S} = \{\text{FT}, \text{LoRA}\}$

Ensure: prediction files, metrics, timing statistics, and error-analysis artifacts

```
1: for all  $b \in \mathcal{B}$  do ▷ Classical sparse baselines
2:   compute TF-IDF features  $\Phi(x)$  for  $x \in D_{\text{train}}$ 
3:    $\psi_b \leftarrow \arg \min_{\psi} \mathcal{L}_{\text{train}}(\psi, \Phi(D_{\text{train}}))$ 
4:    $\hat{y} \leftarrow f_{\psi_b}(\Phi(x))$  for  $x \in D_{\text{val}}$ , evaluate metrics
5: end for
6: for all  $n \in \mathcal{N}$  do ▷ Neural models (from-scratch)
7:   randomly initialize  $\psi_n$ , minimize  $\mathcal{L}$  on  $D_{\text{train}}$  via Eq. (6)
8:    $\psi^* = \arg \min_{\psi} \mathcal{L}(\psi, D_{\text{val}})$ 
9:    $\hat{y} = f_{\psi^*}(x)$  for  $x \in D_{\text{val}}$ , store metrics
10: end for
11: for all  $m \in \mathcal{M}$  do ▷ PLMs (fine-tuning)
12:   for all  $s \in \mathcal{S}$  do
13:     load tokenizer and pretrained checkpoint for  $m$ 
14:     if  $s = \text{LoRA}$  then
15:       inject LoRA adapters  $BA$  via Eq. (4)
16:     else
17:       enable end-to-end updates via Eq. (3)
18:     end if
19:     minimize  $\mathcal{L}$  on  $D_{\text{train}}$  via Eq. (6)
20:      $\psi^* = \arg \min_{\psi} \mathcal{L}(\psi, D_{\text{val}})$ 
21:     compute  $\hat{y} = f_{\psi^*}(x)$  for  $x \in D_{\text{val}}$ , store metrics
22:   end for
23:   for  $k = 1$  to 3 do
24:     partition  $D_{\text{train}} \rightarrow \{D_{\text{train}}^{(k)}, D_{\text{val}}^{(k)}\}$ 
25:      $\psi^{(k)} \leftarrow \text{optimize}(\mathcal{L}, D_{\text{train}}^{(k)}, m)$ 
26:      $\hat{D}_{\text{val}}^{(k)} \leftarrow \{f_{\psi^{(k)}}(x) \mid x \in D_{\text{val}}^{(k)}\}$ 
27:   end for
28:   compute  $\text{Score}_{\text{CV}}$  using Eq. (5)
29: end for
30: aggregate  $\hat{D}_{\text{val}}$  and metrics to produce final artifacts
```

5 Experiments

This section outlines the experimental configuration, baseline selections, and evaluation metrics. We then present a comparative analysis of our findings regarding predictive performance, error patterns, and computational efficiency across the evaluated models.

5.1. Experimental Setup

We begin by specifying the compared systems, the evaluation metrics, and the implementation details required to reproduce the benchmark.

5.1.1. Compared Models

The benchmark combines classical baselines, a lightweight neural baseline, and PLM-based models. The classical group contains Logistic Regression, SVM, and Decision Tree [16]. The neural baseline is TextCNN [17], evaluated under from-scratch training and 3-Fold Cross-Validation. The PLM group contains XLM-RoBERTa [10], mBERT [11], and mBERT-Lao [4], each evaluated with Full Fine-tuning, LoRA [12], and 3-Fold Cross-Validation. This setup creates a controlled comparison between sparse models, a compact neural model, and pretrained transformers on the same Lao review data.

The classical baselines were trained on the same fixed train-validation split used for the fixed-split PLM runs. They use character-level TF-IDF features with an n -gram range of 3-5, `min_df=2`, and at most 50,000 features. Logistic Regression and SVM were implemented through `SGDClassifier` with `log_loss` and `hinge` objectives, respectively, while Decision Tree used `max_depth=40` and `min_samples_leaf=2`. TextCNN uses 128-dimensional trainable embeddings, 128 filters for each kernel size in $\{3, 4, 5\}$, and dropout 0.3, giving the benchmark a compact neural reference that does not depend on pretrained weights.

5.1.2. Evaluation Metrics

We report Accuracy, Precision-macro, Recall-macro, and F_1 -macro, which are standard evaluation measures for text classification and information retrieval tasks [1], [20]. Because the dataset is moderately imbalanced, F_1 -macro is treated as the main comparison metric, while the remaining metrics are used to show overall correctness and class-level balance.

For binary sentiment classification, Accuracy is defined as

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN}. \quad (7)$$

For each class $c \in \{0, 1\}$, let

$$P_c = \frac{TP_c}{TP_c + FP_c}, \quad R_c = \frac{TP_c}{TP_c + FN_c}. \quad (8)$$

The macro-averaged scores reported in this paper are then

$$\text{Precision}_{\text{macro}} = \frac{1}{C} \sum_{c=1}^C P_c, \quad \text{Recall}_{\text{macro}} = \frac{1}{C} \sum_{c=1}^C R_c, \quad (F_1)_{\text{macro}} = \frac{1}{C} \sum_{c=1}^C \frac{2P_c R_c}{P_c + R_c}, \quad (9)$$

where c indexes a sentiment class and $C = 2$ for the negative and positive labels. Here, TP , TN , FP , and FN denote true positives, true negatives, false positives, and false negatives, respectively, while TP_c , FP_c , and FN_c are the corresponding class-specific counts used to compute P_c and R_c for class c .

5.1.3. Implementation Details

The experiments use the final filtered Lao review dataset described in Section 3.4. Logistic Regression, SVM, Decision Tree, and fixed-split TextCNN were trained on the same validation split as the fixed-split PLM runs, while 3-fold cross-validation was applied on the training partition for the PLMs and for TextCNN. The exact hyperparameters are listed in Table III; the main difference across PLM runs is whether the full backbone is updated or only LoRA adapters are trained. Runtime metadata recorded across 14 experiment folders indicate a Tesla T4 GPU with 14.56 GB GPU memory, 12.67 GB RAM, and 1 physical CPU core (2 total cores).

TABLE III
TRAINING CONFIGURATION USED IN THE EXPERIMENTS.

Component	Configuration
PLM setup	25 epochs, max length 128, validation loss
Batch sizes	train 16, eval 32
Optimization	lr 1×10^{-5} , weight decay 0.01, warmup 0.1
Cross-validation	Stratified $K = 3$
LoRA setting	$r = 16$, $\alpha = 32$, dropout 0.1
TextCNN setting	embedding 128, filters 128, kernels 3/4/5, dropout 0.3
Baseline features	TF-IDF char n -grams (3–5), max 50,000

5.2. Results and Discussion

We present the principal performance trend and analyze errors and efficiencies to rationalize the trade-off we observed.

5.2.1. Overall Performance

Taken together, Table IV and Fig. 2 suggest that the suitability of the backbone is more important than moving from sparse to neural baselines, or from LoRA to Full Fine-tuning within the same PLM family. TextCNN is strong in the middle and performs better here than Logistic Regression and achieves an SVM-like performance, yet the best scores still come from XLM-RoBERTa variants. This interpretation is consistent with low-resource sentiment studies, which have largely attributed transfer quality to the choice of language as well as data features, rather than simply more parameters being updated [7], [8], [21]. The benchmark also indicates that sparse baselines are still meaningful for

Lao app reviews, which means this task still contains strong lexical sentiment cues even when pretrained encoders are available.

TABLE IV
OVERALL PERFORMANCE GROUPED BY MODEL FAMILY AND TRAINING STRATEGY.

Model Family	Training Strategy	Acc.	F_1	Prec.	Rec.
XLM-RoBERTa	Full Fine-tuning	0.9664	0.9617	0.9544	0.9702
	LoRA	0.9642	0.9589	0.9539	0.9645
	Cross-Validation	0.9648	0.9597	0.9541	0.9659
mBERT	Full Fine-tuning	0.6969	0.6057	0.6336	0.6012
	LoRA	0.6901	0.6048	0.6253	0.6004
	Cross-Validation	0.6967	0.6137	0.6350	0.6084
mBERT-Lao	Full Fine-tuning	0.9407	0.9328	0.9241	0.9437
	LoRA	0.9326	0.9235	0.9154	0.9335
	Cross-Validation	0.9375	0.9285	0.9228	0.9350
TextCNN	From Scratch	0.9477	0.9403	0.9338	0.9478
	Cross-Validation	0.9339	0.9242	0.9192	0.9298
Classical baselines	Logistic Regression	0.9451	0.9370	0.9322	0.9425
	SVM	0.9505	0.9436	0.9363	0.9522
	Decision Tree	0.8878	0.8721	0.8662	0.8792

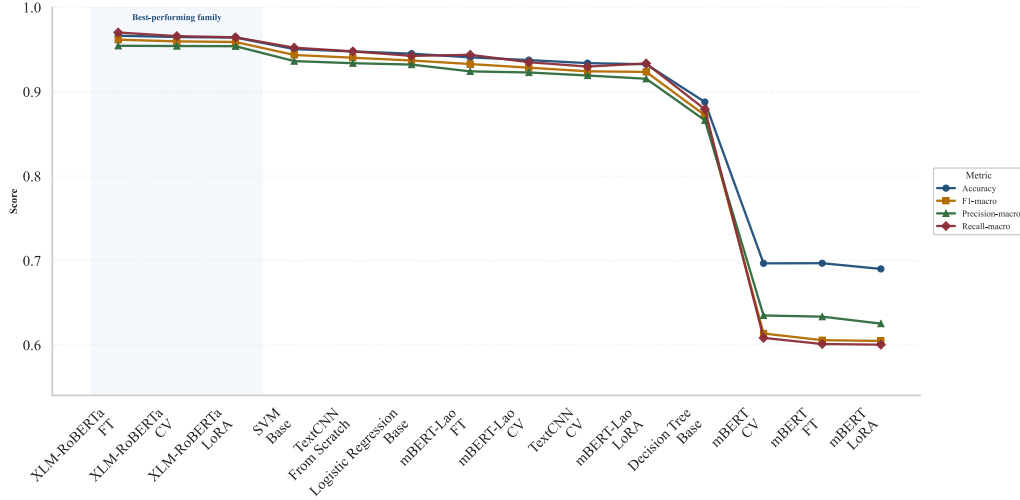


Fig. 2. Metric comparison across all benchmarked settings, sorted by F_1 -macro.

5.2.2. Domain Adaptation Analysis

The gap between mBERT-Lao and generic mBERT suggests that Lao support is not a minor implementation detail but a substantive source of performance difference. This interpretation aligns with LaoPLM and related low-resource work, which argue that multilingual pretraining can under-serve Lao when tokenization behavior and corpus coverage are mismatched [4], [21]. TextCNN further demonstrates that a compact neural model can learn substantive review-local patterns through from-scratch training, achieving competitive performance without the necessity of expensive pre-training stages. However, its slightly lower cross-validation scores relative to its fixed-split peak suggest that from-scratch neural learning remains more sensitive to distribution shifts and training-data variability than the robust representations provided by the strongest PLMs. At the same time, the competitiveness of SVM and Logistic Regression indicates that many app-review decisions are driven by local lexical cues and short formulaic complaints. In other

words, domain adaptation in this dataset appears to depend on both language-aware representations and sensitivity to concise, review-specific expressions.

5.2.3. Error Analysis

Fig.3 helps explain why the model ranking separates so clearly. The strongest systems maintain a more balanced error profile, whereas weaker multilingual transfer is associated with a clear positive-class bias. This matters because the difficult cases are not random; they are concentrated in short complaint-like comments, code-mixed fragments, and mixed-polarity reviews that combine a problem report with a request, update, or partially positive tone. Related low-resource sentiment studies likewise report that short, informal, and mixed-language comments are a recurring source of ambiguity [7], [8], but the present results suggest that these cases are especially consequential for Lao app reviews, where limited context and informal phrasing amplify ambiguity.

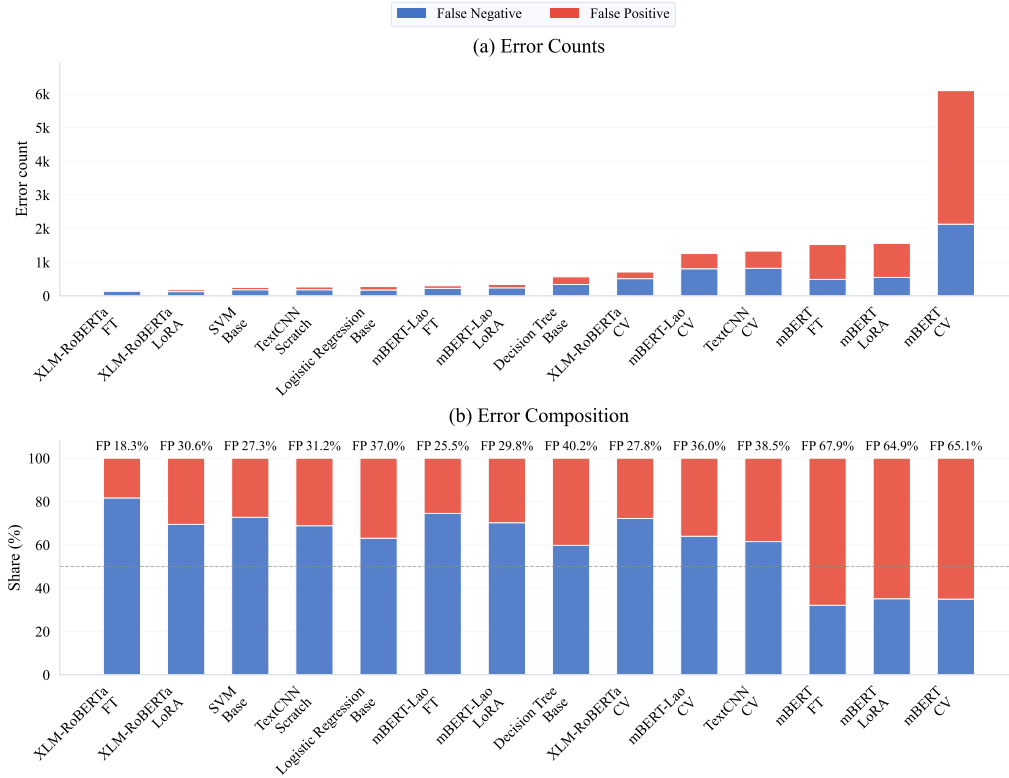


Fig. 3. Error-analysis overview based on the aggregated false-positive and false-negative samples from all experiments. Panel (a) reports raw false-negative and false-positive counts, while panel (b) shows the corresponding error composition percentages for the same experiments.

Table V illustrates this pattern qualitatively. False positives are usually short complaints whose negative intent is expressed indirectly or through code-mixed wording, whereas false negatives more often contain softened criticism, requests, or mixed cues. The dominant failure mode is therefore semantic ambiguity in short feedback, rather than annotation instability alone.

TABLE V
REPRESENTATIVE EXAMPLES OF FIXED-SPLIT ERROR PATTERNS

False Positive		False Negative	
Example	Label	Example	Label
ບໍ່ ok not OK	0	ເປັນຍັງຄື ທັງຫມົດທີ່ຈຶ່ງບໍ່ໄດ້ why does nothing work?	1
ປ່ຽນໂທລະສັບໃໝ່ changed to a new phone	0	ດີກຢູ່ ແຕ່ໃຫ້ອັບເດດດູໂພດ it is good, but please update it	1
ເບີລະຫັດ whatsapp WhatsApp verification code	0	ແຕ່ໃຫ້ປັບປຸງໃຫ້ໄວກ່ວາເກົ່າ please make it faster	1
bug ຫລາຍ too many bugs	0	ໂຄດດີ ຖ້າບໍ່ອັບເດດແອບ ແຮງຊີດີ the code is good, but the update hurts the app	1
ແອັບຊ້າສຸດ the app is very slow	0	ຂອບເຂດສົ່ງ ກ້ວາງໂກລ ສົ່ງໄວ ບໍ່ມີບັນຫາ wide service area, fast delivery, no problems	1

Fig.4 provides a complementary phrase-level view of the same error set using the 30 most frequent short false-positive and false-negative samples. This visualization is intended as a qualitative summary of recurring failure expressions rather than as a replacement for the quantitative counts in Fig.3.



Fig. 4. Phrase-level word clouds built from the 30 most frequent short false-positive and false-negative samples in the fixed-split experiments. Larger phrases indicate more frequently repeated failure cases.

5.2.4. Practical Implications

From a deployment perspective, the results support a tiered selection strategy. When predictive quality is the main priority, XLM-RoBERTa is the most reliable choice. When computational efficiency and localized learning are prioritized over massive parameter transfer, the from-scratch TextCNN model offers a useful middle ground: it reaches 0.9403 F_1 -macro on the fixed split in only 11.28 minutes. Within the PLM family, LoRA offers a better trade-off than retraining the full model. Although this study does not report dedicated inference latency, the much smaller trainable footprint of LoRA suggests lower adaptation and deployment overhead than full fine-tuning in resource-constrained settings. When infrastructure or maintenance constraints dominate, SVM remains a credible baseline because it is stable, interpretable, and inexpensive to train. More broadly, the benchmark suggests that practical Lao sentiment systems should be selected not only by raw accuracy, but also by how well they balance language fit, efficiency, and operational simplicity.

TABLE VI
EFFICIENCY COMPARISON BETWEEN FULL FINE-TUNING (FT) AND LoRA.

Model Family	Strat.	Params	Ratio	Epoch (s)	Total (min)	F_1
XLM-RoBERTa	FT	278.0M	1.0000	267.3	111.4	0.9617
	LoRA	1.48M	0.0053	117.8	49.1	0.9589
mBERT	FT	177.9M	1.0000	205.8	85.8	0.6057
	LoRA	0.89M	0.0050	116.9	48.7	0.6048
mBERT-Lao	FT	124.6M	1.0000	177.5	74.0	0.9328
	LoRA	1.48M	0.0117	113.7	47.4	0.9235

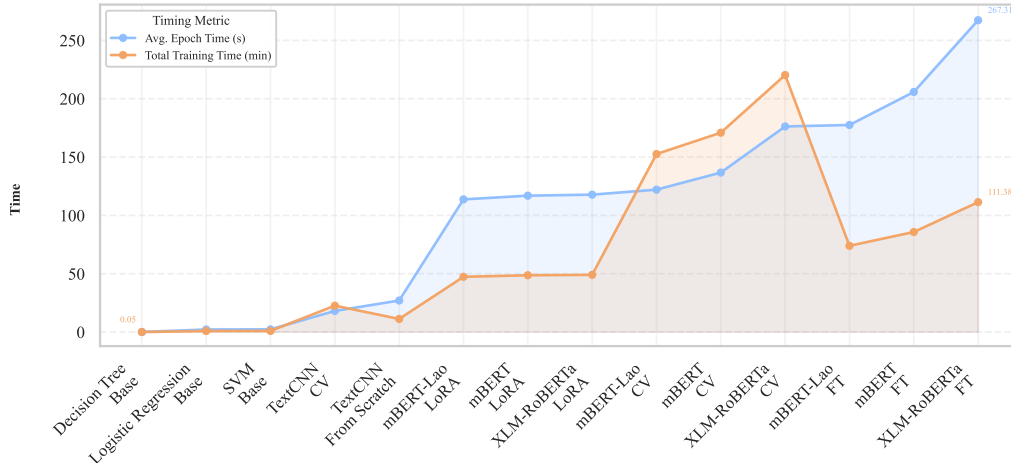


Fig. 5. Timing comparison across all benchmarked settings, sorted by average epoch time.

6 Conclusion

A comprehensive benchmark for Lao sentiment classification was set up by using a large-scale, filtered dataset of real-world mobile application reviews in this study. We show in our systematic evaluation across different models, spanning classical sparse baselines, from-scratch neural references as well as best-of-breed PLMs that while XLM-RoBERTa delivers the most robust performance, the choice of adaptation strategy and language support are essential components in Lao NLP. In particular we show that (i) monolingual pre-training and cross-lingual transfer (XLM-RoBERTa) routinely outperform those based on generic multilingualism; (ii) using LoRA has good quality at minimal computational overhead; (iii) lightweight from-scratch neural training (TextCNN) gives an authoritative reference that can learn review-specific patterns without PLM weights. While doing so, the study also notes significant problems surrounding linguistic ambiguity, noise, and semantic shifts within code-mixed spaces.

In addition, we recommend that future research work primarily on diversity of datasets and consistency for labels while examining more granular techniques for studying aspect-level sentiment in order to gain a clearer grasp on users' experience. We aim to help scale up and train more precise NLP tools in the Lao digital ecosystem by providing the Lao digital ecosystem with our dataset and trained models.

References

- [1] K. L. Tan, C. P. Lee, and K. M. Lim, "A survey of sentiment analysis: Approaches, datasets, and future research," *Applied Sciences*, vol. 13, no. 7, p. 4550, 2023.
- [2] K. Alahmadi, S. Alharbi, J. Chen, and X. Wang, "Generalizing sentiment analysis: a review of progress, challenges, and emerging directions," *Social Network Analysis and Mining*, vol. 15, no. 1, p. 45, 2025.

- [3] Q.-V. Pham, N.-H. Dang, P.-L. Tran, and Q.-H. Le, "Fine-tuning pre-trained language models for vietnamese sentiment classification in e-commerce," in *Computational Intelligence in Engineering Science*, N. T. Nguyen, A.-C. Le, T. D. Tran, T.-P. Hong, Y. Manolopoulos, N. A. Le Khac, P. Tran Tin, and V.-N. Huynh, Eds. Cham: Springer Nature Switzerland, 2026, pp. 160–174.
- [4] N. Lin, Y. Fu, C. Chen, Z. Yang, and S. Jiang, "Laoplms: Pre-trained language models for lao," in *Proceedings of the Thirteenth Language Resources and Evaluation Conference*, 2022, pp. 6506–6512.
- [5] H. T. Nguyen, T. Palongve, C. Q. Nguyen, and K. T. Le, "Semi-automatic construction and benchmarking of a word-segmented corpus for lao using llms and transformer models," *Informatica*, vol. 49, no. 27, 2025.
- [6] W. Phatthiyaphaibun, "Laonlp: Lao language natural language processing," Jul. 2022. [Online]. Available: <https://doi.org/10.5281/zenodo.6833407>
- [7] M. Subramanian, K. Shanmugavadevel *et al.*, "Kec_tech_titans@ dravidianlangtech 2025: Sentiment analysis for low-resource languages: Insights from tamil and tulu using deep learning and machine learning models," in *Proceedings of the Fifth Workshop on Speech, Vision, and Language Technologies for Dravidian Languages*, 2025, pp. 278–282.
- [8] M. Chowdhury, A. Laskar, T. Ahmad, and A. T. Wasi, "Mysticciol@ dravidianlangtech 2025: A hybrid framework for sentiment analysis in tamil and tulu using fine-tuned sbert embeddings and custom mlp architectures," in *Proceedings of the Fifth Workshop on Speech, Vision, and Language Technologies for Dravidian Languages*, 2025, pp. 167–172.
- [9] T. Durairaj, B. R. Chakravarthi, A. Hegde, H. L. Shashirekha, R. Natarajan, S. Thavareesan, R. Sakuntharaj, C. Rajkumar, P. Shetty, H. S. Kumar *et al.*, "Overview of the shared task on sentiment analysis in tamil and tulu," in *Proceedings of the Fifth Workshop on Speech, Vision, and Language Technologies for Dravidian Languages*, 2025, pp. 732–738.
- [10] A. Conneau, K. Khandelwal, N. Goyal, V. Chaudhary, G. Wenzek, F. Guzmán, E. Grave, M. Ott, L. Zettlemoyer, and V. Stoyanov, "Unsupervised cross-lingual representation learning at scale," in *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*, 2020, pp. 8440–8451.
- [11] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, "Bert: Pre-training of deep bidirectional transformers for language understanding," in *Proceedings of the 2019 conference of the North American chapter of the association for computational linguistics: human language technologies, volume 1 (long and short papers)*, 2019, pp. 4171–4186.
- [12] E. J. Hu, Y. Shen, P. Wallis, Z. Allen-Zhu, Y. Li, S. Wang, L. Wang, and W. Chen, "Lora: Low-rank adaptation of large language models," in *International Conference on Learning Representations*, 2022.
- [13] X. Han, Z. Zhang, N. Ding, Y. Gu, X. Liu, Y. Huo, J. Qiu, Y. Yao, A. Zhang, L. Zhang *et al.*, "Pre-trained models: Past, present and future," *AI open*, vol. 2, pp. 225–250, 2021.
- [14] B. Min, H. Ross, E. Sulem, A. P. B. Veyseh, T. H. Nguyen, O. Sainz, E. Agirre, I. Heintz, and D. Roth, "Recent advances in natural language processing via large pre-trained language models: A survey," *ACM Computing Surveys*, vol. 56, no. 2, pp. 1–40, 2023.
- [15] S. Ruder, N. Constant, J. Botha, A. Siddhant, O. Firat, J. Fu, P. Liu, J. Hu, D. Garrette, G. Neubig *et al.*, "Xtreme-r: Towards more challenging and nuanced multilingual evaluation," in *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing*, 2021, pp. 10 215–10 245.
- [16] F. Pedregosa, G. Varoquaux, A. Gramfort, V. Michel, B. Thirion, O. Grisel, M. Blondel, P. Prettenhofer, R. Weiss, V. Dubourg *et al.*, "Scikit-learn: Machine learning in python," *the Journal of machine Learning research*, vol. 12, pp. 2825–2830, 2011.
- [17] Y. Kim, "Convolutional neural networks for sentence classification," in *Proceedings of the 2014 conference on empirical methods in natural language processing (EMNLP)*, 2014, pp. 1746–1751.
- [18] Y. Mao, Y. Ge, Y. Fan, W. Xu, Y. Mi, Z. Hu, and Y. Gao, "A survey on lora of large language models," *Frontiers of Computer Science*, vol. 19, no. 7, p. 197605, 2025.
- [19] S. Nwaiwu, "Parameter-efficient fine-tuning for low-resource text classification: a comparative study of lora, ia3, and reft," *Frontiers in big Data*, vol. 8, p. 1677331, 2025.
- [20] J. Opitz, "A closer look at classification evaluation metrics and a critical reflection of common evaluation practice," *Transactions of the Association for Computational Linguistics*, vol. 12, pp. 820–836, 2024.
- [21] N. Z. Lamin and A. A. Aziz, "Cross-lingual sentiment analysis in low-resource languages: A recent review on tasks, methods and challenges," *International Journal of Advanced Computer Science & Applications*, vol. 16, no. 11, 2025.